Google Groups Management & Governance (Types, Lifecycle, Membership & Security)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

1. Solution Overview — Google Groups Management & Governance 4

The Groups model — six design decisions, one governed estate . . . 5

Reference design (target state) . . . 7

Benefits . . . 7

Limitations & honest constraints . . . 7

Deployment model . . . 8

2. Requirements 8

2.1 Super-admin (or delegated Groups admin) with Groups & Cloud Identity rights . . . 9

2.2 Cloud Identity Premium (for dynamic groups & security groups) . . . 9

2.3 Group-type catalogue & naming standard (agreed and documented) . . . 9

2.4 Directory attribute source of truth (for dynamic groups) . . . 10

2.5 Groups lifecycle owners & request/approval path . . . 10

2.6 API access + least-privilege scopes for the Python tools . . . 10

Global Requirements — Google Workspace & Gemini Solutions 11

1. Google Workspace / Cloud Identity tenant . . . 11

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 11

3. DNS control for the routed domains . . . 12

4. Identity & provisioning source of truth . . . 12

5. A pilot scope (OU / group) and change control . . . 12

6. Security, compliance & governance stakeholders . . . 13

3. Implementation 13

Stage 0 — Assess (read-only baseline) . . . 13

Stage 1 — Design (agree the model) . . . 14

Stage 2 — Configure the org-wide baseline . . . 15

Stage 3 — Pilot (apply the model to the pilot scope) . . . 15

Stage 4 — Validate the pilot . . . 16

Stage 5 — Expand & operationalise . . . 17

5. Testing & Validation 17

Test 1 — Baseline captured (Gate 0) . . . 17

Test 2 — Org-wide baseline configured (Gate 2) . . . 18

Test 3 — Naming & type conformance (Gates 3/4) . . . 18

Test 4 — Membership & external-exposure policy (Gates 3/4) . . . 18

Test 5 — Ownership & no-orphan (Gates 3/4) . . . 18

Test 6 — Stale-group hygiene . . . 19

Test 7 — Dynamic membership correctness . . . 19
Test 8 — Security-group access re-point . . . 19
Test 9 — Lifecycle review report generation . . . 19
Test 10 — Rollback rehearsal (high-impact changes) . . . 20
6. Troubleshooting 20
Auth, scopes & API access . . . 20
Naming/type validator failures (Gate 3/4) . . . 20
Settings that "won't stick" / apply has no effect . . . 21
External-member removal breaks mail flow or collaboration . . . 21
Dynamic-group membership is wrong or empty . . . 22
Ownership / orphaned groups . . . 22
Security-group / Cloud Identity issues . . . 22
Retirement pitfalls . . . 22
Rate limits / large estates . . . 22
7. Best Practices 23
Model & typing . . . 23
Ownership & membership . . . 23
Exposure & trust boundary . . . 23
Lifecycle & hygiene . . . 24
Automation & change safety . . . 24
8. Deliverables Summary 24
Documentation . . . 24
The 5 Python tools . . . 25
Appendix A — PowerShell Scripts 27

Executive Summary

Google Groups are load-bearing — and almost never governed. A single group is simultaneously a mailing address, an access-control principal (added to Shared Drives, calendars, IAM, and SSO apps), and a collaboration forum. Because any user can create one, they multiply faster than anyone tracks: groups no one remembers the purpose of, groups whose only owner has left the company, groups still holding external members from a finished project, and groups open enough that anyone can post or read the archive. Each is an uncontrolled distribution path and an unaudited access grant.

This solution puts Google Groups under a real management and governance model so they are **intentional, least-exposed, and auditable**. It establishes a fixed set of **group types** with a **naming convention**, a **lifecycle** (request → approve → review → retire), a **membership and ownership model** (≥2 owners, owner *groups*, group-based nested membership), the right use of **dynamic groups** and **Cloud Identity security groups**, a restrictive **external-membership and posting/access-settings policy**, and continuous **stale/orphaned-group hygiene**.

Outcome. Every group is typed and named to standard; no group is orphaned or ownerless; external exposure is scoped to explicitly external groups with allow-listed partners; access-granting groups carry the immutable security label; and the whole estate is machine-checkable against policy and reviewed on a risk-based cadence — control restored, and kept.

Approach — why it's low-risk. The work follows a disciplined, pilot-first model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with an explicit pass/fail gate at every stage and a **documented rollback** for every high-impact change (tightening settings, removing external members, retiring a group is staged suspend → hold → delete, never a single destructive step). Five included **Python tools** make it repeatable and evidence-based: they are **read-only or dry-run by default**, carry **no credentials in code**, and act as the validation gates. Nothing is changed tenant-wide until it has passed on one pilot scope.

Bottom line. Google Groups made intentional and auditable — typed and named to standard, lifecycle-governed, membership and external exposure scoped, no stale or orphaned groups — delivered self-guided and tested, at a fraction of the cost of a consulting engagement.

1. Solution Overview — Google Groups Management & Governance

In a Google Workspace enterprise, **Groups are load-bearing infrastructure that almost nobody governs**. A single Google Group is, at once, a **mailing list** (a distribution address), an **access-control principal** (added to Shared Drives, calendars, IAM roles, target audiences, and SSO apps), and a **collaboration space** (a Groups-for-Business forum). Because a group is so cheap to create — any user can spin one up from groups.google.com by default — they multiply faster than any governance catches up. The result is predictable: thousands of groups where nobody remembers the purpose, groups whose only owner left the company (**orphaned**), groups that still hold **external members** from a project that ended two years ago, and groups configured so **anyone on the internet can post** or **anyone in the org can join and read** the archive. Every one of those groups is a distribution path, an access grant, and an audit finding waiting to happen.

This solution establishes a **management and governance model for Google Groups** so groups are **intentional, least-exposed, and auditable**: a defined set of **group types** with a **naming convention**, a **lifecycle** (request → approve → review → retire), a **membership and ownership model**, the right use of **dynamic groups** and **Cloud Identity security groups**, an **external-membership and posting/access-settings policy**, and a recurring **stale/orphaned-group hygiene** discipline. It is deployed **pilot-first**, with pass/fail gates and a documented rollback for the changes that are high-impact (tightening posting/access settings and removing external members can break active mail flow and collaboration).

> ***Scope boundary — governance model, not a one-time migration.*** *This solution governs the* ***steady-state Groups estate****. Bulk *creation* of groups from a migrated distribution-list export (Exchange DLs → Google Groups) is a migration-execution activity and is referenced only where the target type/naming/policy must be agreed* ***before*** *groups are loaded. This document assumes groups exist (or are landing) and defines how they are typed, named, owned, scoped, reviewed, and retired from that point on.*
>
> ***Vendor boundary.*** *Google* ***Admin console*** *(admin.google.com) paths, the* ***Groups settings*** *surface,* ***Cloud Identity dynamic/security groups****, and the* ***Admin SDK Directory (Groups)*** *+* ***Cloud Identity Groups*** *APIs used by the Python tools are authored from Google's current public documentation. Several capabilities are* ***edition-gated*** *—* ***dynamic groups*** *and* ***Cloud Identity security groups*** *require* ***Cloud Identity Premium*** *(bundled with Enterprise Workspace tiers) — and those gates are flagged where they apply. Where a console page or toggle is evolving it is flagged* ***"verify in your tenant."*** *Compliance *interpretation* (what an auditor will accept as evidence, which regime applies) is deferred to the organization's compliance/legal owners; this solution provides the technical control, configuration method, and evidence.*

The Groups model — six design decisions, one governed estate

A governed Groups estate is the product of six deliberate design decisions. The reference model below is what the Implementation Guide stands up.

*Decision 1 — Group types (what a group is *for*)*

Not all groups are the same object, and conflating them is the root of most sprawl. The design defines a small, fixed set of **types**, each with its own default policy:

• **Distribution group** — a mail address for one-to-many email (e.g. `all-finance@`). Members receive mail; the group is rarely an access principal.

• **Access/security group** — a principal used to *grant access* (Shared Drives, calendars, IAM, target audiences, SSO app assignment). Mail delivery is secondary or off. Best implemented as a **Cloud Identity security group** so it carries the immutable security label.

• **Collaboration group** — a Groups-for-Business forum/inbox (shared inbox, team discussion, Collaborative Inbox) where members read and post to a shared archive.

• **Dynamic group** — membership is computed from a **CEL query** over directory attributes (e.g. all users in `department=Finance`) rather than managed by hand.

• **External/partner group** — deliberately contains members outside the organization; governed under

the strictest external policy. The type drives every downstream default (who can join, who can post, external allowed, review cadence).

Decision 2 — Naming convention (make the type and owner obvious)

A group's name should announce its type, scope, and purpose so an admin never has to guess. The design adopts `<Type> – <Scope> – <Purpose>` for the display name and a matching **email prefix convention** (e.g. `dl-`, `grp-`/`sec-`, `collab-`, `dyn-`, `ext-`). Examples: `Distribution – Finance – All Staff` at `dl-finance-all@corp.example`; `Security – Finance – AP Drive Access` at `sec-finance-ap@corp.example`; `Dynamic – Sales – All Sales` at `dyn-sales-all@corp.example`; `External – ProjectX – Partner Collab` at `ext-projectx-partner@corp.example`. The prefix makes type-based policy validation and reporting trivial (a script can assert every `sec-*` group is a security group with no external members).

Decision 3 — Membership & ownership model (who is in, who is accountable)

Every group must have a **named, accountable owner** — and never a single point of failure. The design requires **at least two owners per group** (ideally an **owner *group***, not two individuals, so ownership survives departures), assigns the `OWNER` / `MANAGER` / `MEMBER` roles deliberately, and prefers **group-based (nested) membership** over long lists of individual users so access is auditable in one place and driven by the directory source of truth. An **orphaned group** — zero live owners — is a defect the lifecycle must catch and fix.

Decision 4 — Dynamic vs. static, and Cloud Identity security groups

For membership that maps cleanly to a directory attribute (department, location, cost centre), a **dynamic group** (Cloud Identity Premium) keeps membership correct automatically and eliminates the "someone forgot to remove them" class of error — the tradeoff is that membership is only as good as the attribute source of truth (HR/IGA feeding the directory). For any group used to **grant access**, make it a **Cloud Identity security group** so it carries the immutable **security label**: security groups can be used in **IAM and access contexts** where plain Groups-for-Business groups cannot, and the label prevents the group from being silently repurposed. The design decides, per group, static-vs-dynamic and whether it must be a security group.

Decision 5 — External-membership + posting/access-settings policy (the trust boundary)

This is where a group leaks. Every group carries **Groups settings** that determine **who can join**, **who can post/message**, **who can view the archive/members**, and **whether members outside the organization are allowed**. Left at permissive defaults, a group can be **public** (anyone can post, external senders accepted, archive world-readable). The design sets a **restrictive baseline at the Admin-console level** (org-wide default access, "who can create groups", and whether external members are allowed at all) and a **per-type policy** applied to each group's settings — internal-only posting, owner-approved joins, members-only archive, external members only on explicitly `ext-*` groups. The default posture is **restrictive**, opened deliberately per type.

Decision 6 — Lifecycle & hygiene (keeping it governed over time)

Groups decay. The design stands up a **lifecycle** — **request** → **approve** → **review** → **retire** — so groups are created to the model rather than ad hoc, reviewed on a cadence, and retired when their purpose ends. Hygiene sweeps run continuously: an **orphaned-group** finder (no live owner), a **stale-group** finder (no posts / no membership change / empty for N days), an **external-exposure** finder (external members or public settings outside policy), and an **over-permissioned** finder (security-type groups whose settings are

too open). Retirement is **staged** (suspend delivery / archive → hold → delete) with a rollback window, because deleting a group that still routes mail is destructive.

Reference design (target state)

Layer	Target state
Types	Fixed set: Distribution, Security/Access, Collaboration, Dynamic, External — each with a default policy
Naming	`<Type> – <Scope> – <Purpose>` display + email prefix (`dl-/sec-/collab-/dyn-/ext-`)
Ownership	≥2 owners, ideally an **owner group**; no orphaned groups; group-based nested membership
Dynamic/Security	Dynamic groups for attribute-driven membership; **Cloud Identity security groups** for anything granting access
External / posting	Restrictive Admin baseline; per-type settings; external members only on `ext-*` groups; internal-only posting default
Lifecycle	Request → approve → review → retire; owner-attested access reviews; staged retirement with rollback
Hygiene	Continuous orphaned / stale / external-exposed / over-permissioned sweeps

Benefits

• **No orphaned or ownerless groups** — every group has ≥2 accountable owners; the lifecycle catches and fixes any that lose them.

• **Least exposure** — restrictive posting/access defaults and a hard external boundary keep groups from becoming open relays or world-readable archives.

• **Membership stays correct** — dynamic groups track directory attributes; group-based membership is auditable in one place.

• **Access is trustworthy** — Cloud Identity security groups carry an immutable label, so access grants can't be silently repurposed.

• **Auditable & scales** — a naming convention plus type policy makes the whole estate machine-checkable;

lifecycle and hygiene sweeps keep it governed as it grows.

Limitations & honest constraints

• **Edition-gated controls.** Dynamic groups and Cloud Identity security groups require **Cloud

Identity Premium**; on lower tiers the design falls back to static groups + manual review (flagged in Requirements).

• **Dynamic membership is only as good as its source.** A dynamic group is correct only if the directory attribute it queries is correct — garbage attributes yield wrong membership; the HR/IGA → directory feed is a prerequisite, not an afterthought.

• **Tightening settings can break things.** Restricting posting, removing external members, or changing who can post to a distribution address can break active mail flow and collaboration; every production settings change is piloted, gated, and rollback-ready.

• **Existing sprawl must be remediated.** This design defines the target and the sweeps to reach it; re-typing, re-naming, re-owning, and cleaning up thousands of existing groups is an execution effort, not a single switch.

• **Retirement is destructive.** Deleting a group that still routes mail or grants access breaks both; retirement is staged (suspend → hold → delete) with a rollback window.

Deployment model

This solution follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit pass/fail gates and a **documented rollback** for every production settings change:

1. **Assess** — inventory the current Groups estate and its risk posture (orphaned, stale, external-exposed, over-permissioned) read-only with `groups_inventory_hygiene_assessment.py`.
1. **Design** — agree the type set, naming convention, ownership model, dynamic/security-group decisions, external/posting policy, and lifecycle against the reference design above.
1. **Configure** — set the Admin-console Groups baseline, define the named policies, and prepare the pilot groups to the naming/type model (dry-run first with the config/settings helpers).
1. **Pilot** — apply the model to one pilot scope (e.g. `Pilot OU – Groups – Finance`) with a defined pass/fail gate.
1. **Validate** — confirm naming/type policy is enforced, no orphaned/stale groups remain in scope, and external exposure is scoped, using the validator + policy-checker tools.
1. **Expand** — roll the model out scope by scope, keeping the lifecycle and hygiene sweeps as steady-state governance.

2. Requirements

These are the **solution-specific** prerequisites for Google Groups management & governance. They build on the shared **[Global Requirements — Google Workspace & Gemini Solutions](global-requirements-google.md)** (Workspace/Cloud Identity tenant with super-admin,

identity source of truth, a pilot scope + change control, and security/compliance stakeholders) — those are assumed and are **not** repeated here. Verify each item below before starting Section 3.

2.1 Super-admin (or delegated Groups admin) with Groups & Cloud Identity rights

• **Description:** A **super-admin**, or a custom admin role granting the **Groups** privileges (create/read/update/delete groups, manage group settings and membership) plus **Cloud Identity** rights for security/dynamic groups. API automation uses the **Admin SDK Directory (Groups, Members, Groups Settings)** and **Cloud Identity Groups** APIs.

• **Why required:** All group typing, naming, settings, ownership, dynamic/security-group configuration, and lifecycle actions are performed with these privileges.

• **How to verify:** In **admin.google.com** → **Account** → **Admin roles**, confirm the role holds the Groups and Cloud Identity privileges; confirm you can open **Directory** → **Groups** and edit a group's **Settings** and **Members**.

• **How to implement if missing:** Assign the built-in **Groups Admin** role plus Cloud Identity privileges, or a custom role scoped to the above, per least privilege.

2.2 Cloud Identity Premium (for dynamic groups & security groups)

• **Description: Cloud Identity Premium** licensing (bundled with Enterprise Workspace editions, or purchasable standalone) enabling **dynamic groups** and **Cloud Identity security groups**.

• **Why required:** Attribute-driven (dynamic) membership and the immutable **security label** used for access-granting groups are Premium features; without them the design falls back to static groups and manual review.

• **How to verify:** In **admin.google.com** → **Billing** → **Subscriptions**, confirm a Cloud Identity Premium (or equivalent Enterprise) subscription; attempt to create a test **dynamic group** in **Directory** → **Groups** → **Create group** (the dynamic option is present only with Premium — **verify in your tenant**).

• **How to implement if missing:** Acquire Cloud Identity Premium, or document that dynamic/security groups are out of scope for this tenant and use the static-group fallback throughout.

2.3 Group-type catalogue & naming standard (agreed and documented)

• **Description:** A written, stakeholder-agreed **type catalogue** (Distribution / Security-Access / Collaboration / Dynamic / External) with the **default policy per type**, and the **naming convention** (`<Type> – <Scope> – <Purpose>` display + email prefixes `dl-`/`sec-`/`collab-`/`dyn-`/`ext-`).

• **Why required:** Every configuration step, the naming/type validator tool, and all reporting key off this standard; without it, "governed" has no definition to enforce against.

• **How to verify:** Confirm a document exists naming the types, their default settings, and the exact

naming/prefix rules, approved by the Groups/security owners.

• **How to implement if missing:** Author the type catalogue and naming standard from the reference design in Section 1 before running the validators; keep it as the source of truth the tools check against.

2.4 Directory attribute source of truth (for dynamic groups)

• **Description:** A **reliable, populated set of directory attributes** (e.g. `department`, `costCenter`, `orgUnitPath`, `location`) fed from the **HR/IGA source of truth** via the provisioning mechanism named in Global Requirement 4 (GCDS/SCIM), for any group whose membership will be **dynamic**.

• **Why required:** A dynamic group's membership is a **CEL query over these attributes** — if they are empty, stale, or inconsistent, the computed membership is wrong (and silently so).

• **How to verify:** In **Directory** → **Users**, spot-check that the attributes your CEL queries reference are populated and current for a representative sample; confirm the sync that maintains them is healthy.

• **How to implement if missing:** Populate and reconcile the attributes at the source and let the sync flow them into the directory **before** converting any group to dynamic; until then, keep those groups static.

2.5 Groups lifecycle owners & request/approval path

• **Description:** Named **owners of the Groups lifecycle** and an agreed **request** → **approve** intake path (a form/ticket queue) and **review/retire** cadence, plus the **change-control window** (from Global Requirement 5) for production settings changes.

• **Why required:** Governance is a process, not a one-time config: someone must approve new groups to the model, run access reviews, and authorise staged retirements; posting/external changes are high-impact and need change control.

• **How to verify:** Confirm a named intake queue/owner exists, a review cadence is agreed, and a change window/approval path is available for Groups settings changes.

• **How to implement if missing:** Stand up the intake queue (form/ticket), assign lifecycle owners, and agree the review cadence and change-control process before enforcing the model in production.

2.6 API access + least-privilege scopes for the Python tools

• **Description:** An authorized way for the Python tools to call Google APIs **without secrets in code**: **either** Application Default Credentials (ADC) **for an authenticated admin, or a** service account with domain-wide delegation **impersonating an admin, granted only the scopes the tools need. Enable the** Admin SDK API **and** Cloud Identity API** on the project.

• **Why required:** The assessment/validation tools read Groups, members, and settings; the helpers

(with `--apply`) write settings/membership. Least-privilege, secret-free auth is mandatory.

- **Read-only scopes (assessment/validation):** `admin.directory.group.readonly`, `admin.directory.group.member.readonly`, `apps.groups.settings` (read), `cloud-identity.groups.readonly`, `admin.directory.user.readonly`.
- **Write scopes (only for `--apply` helpers, granted separately):** `admin.directory.group`, `admin.directory.group.member`, `apps.groups.settings`, `cloud-identity.groups`.
- **How to verify:** Confirm the APIs are enabled, the ADC/service-account identity resolves, and a read-only tool run (e.g. `groups_inventory_hygiene_assessment.py`) returns data; confirm no key file is committed to the repo.
- **How to implement if missing:** Enable the APIs, configure ADC or DWD with the read-only scopes above, and add the write scopes only to the identity that runs the `--apply` helpers under change control.

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

- **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).
- **Why required:** All configuration, policy, and API work is performed against this tenant.
- **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.
- **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

- **Description:** For migration/coexistence/identity solutions, administrative access to the source

Microsoft 365 tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the

owners before making production changes.

6. Security, compliance & governance stakeholders

- **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance**

stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

- **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to

enterprise policy and legal requirements, not just technical defaults.

- **How to verify:** Confirm the applicable policies and named stakeholders/approvers.
- **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements

before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

3. Implementation

Follow these steps in order. They implement the reference design from Section 1 using the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit **pass/fail gates** and a **documented rollback** for every high-impact change (posting/access-settings tightening, external-member removal, retirement). **Every artifact is named** with `<Type> – <Scope> – <Purpose>` (or the convention for that object) and the **exact name is reused** in later steps, testing, troubleshooting, and the scripts. Navigation is written for the current Google Admin console; where a page is evolving it is flagged **"verify in your tenant."**

Throughout, `corp.example` stands in for the organization's primary domain.

Stage 0 — Assess (read-only baseline)

Step 0.1 — Inventory the Groups estate and its risk posture. Run the read-only assessment to get a baseline before changing anything:

```
python groups_inventory_hygiene_assessment.py --domain corp.example --json > baseline-groups.json
```

It enumerates every group and flags **ORPHANED** (no live owner), **STALE** (empty / no recent activity), **EXTERNAL-EXPOSED** (external members or public settings), **OVER-PERMISSIONED** (access-granting group with open settings), and **NAMING-NONCONFORMANT** (name/prefix off-standard). Save the output as artifact `Baseline – Groups Estate – Assess`.

Step 0.2 — Record the current Admin-console Groups baseline. In **admin.google.com** → **Directory** → **Groups** → **Settings** (org-wide **sharing/settings** for groups) and **Apps** → **Google Workspace** → **Groups for Business**, note the current values for: **who can create groups**, **default access level**, **allow external members**, **who can post to groups from outside the organization**. This is your **rollback reference** — capture it before any change.

> ***Gate 0 (baseline captured):*** *you have* `Baseline – Groups Estate – Assess` *and the recorded Admin-console settings.* ***Pass*** *→ proceed to Design. Do not change settings until the baseline exists.*

Stage 1 — Design (agree the model)

Step 1.1 — Finalise the type catalogue and per-type policy. Confirm the five types and lock the **default settings per type** (see table). This table is the policy the validators and helpers enforce.

Type (prefix)	whoCanJoin	whoCanPostMessage	whoCanViewGroup	whoCanViewMembership	External members	Security label
Distribution (`dl-`)	`INVITED_CAN_JOIN`	`ALL_IN_DOMAIN_CAN_POST`	`ALL_IN_DOMAIN_CAN_VIEW`	`ALL_MANAGERS_CAN_VIEW`	No	Optional
Security/Access (`sec-`)	`INVITED_CAN_JOIN`	`NONE_CAN_POST`	`ALL_MANAGERS_CAN_VIEW`	`ALL_MANAGERS_CAN_VIEW`	No	**Required**
Collaboration (`collab-`)	`CAN_REQUEST_TO_JOIN`	`ALL_MEMBERS_CAN_POST`	`ALL_MEMBERS_CAN_VIEW`	`ALL_MEMBERS_CAN_VIEW`	No	Optional
Dynamic (`dyn-`)	membership computed (CEL)	per underlying type	`ALL_IN_DOMAIN_CAN_VIEW`	`ALL_MANAGERS_CAN_VIEW`	No	Optional
External (`ext-`)	`INVITED_CAN_JOIN`	`ALL_MEMBERS_CAN_POST`	`ALL_MEMBERS_CAN_VIEW`	`ALL_MEMBERS_CAN_VIEW`	**Yes (allow-listed)**	Optional

> *Setting names/values above are Groups Settings API fields; confirm the exact enum values against the Groups Settings API reference for your tenant version —* ***verify in your tenant*** *where a value is deprecated or renamed.*

Step 1.2 — Finalise the naming convention. Lock `<Type> – <Scope> – <Purpose>` display names and the email prefixes. Example canonical artifacts used through the rest of this guide:

- `Distribution – Finance – All Staff` → `dl-finance-all@corp.example`
- `Security – Finance – AP Drive Access` → `sec-finance-ap@corp.example`
- `Collaboration – Finance – AP Team Inbox` → `collab-finance-ap@corp.example`
- `Dynamic – Sales – All Sales` → `dyn-sales-all@corp.example`
- `External – ProjectX – Partner Collab` → `ext-projectx-partner@corp.example`
- Owner group (never individuals): `Security – Groups – Owners` → `sec-groups-owners@corp.example`

Step 1.3 — Design the ownership model. Standard: ≥**2 owners per group**, and prefer an **owner *group*** as the `OWNER` rather than named individuals, so ownership survives departures. Nominate `sec-groups-owners@corp.example` (from Step 1.2) as the default owner group for governed/security groups.

Step 1.4 — Design the lifecycle. Document **request** → **approve** → **review** → **retire**: the intake form/queue (from Requirement 2.5), the approval rule (must specify type, scope, purpose, owner group, external? and be approved to the type catalogue), the **review cadence** (owner-attested, e.g. quarterly for `sec-`/`ext-`, annually for `dl-`/`collab-`), and the **staged retirement** procedure (Stage 5).

Step 1.5 — Choose the pilot scope. Select one scope to pilot on and name it `Pilot OU – Groups – Finance` — the Finance function's groups (or an equivalent). All Stage-3/4 changes are made here first.

> ***Gate 1 (design signed off):*** *type catalogue, naming convention, ownership model, lifecycle, and named pilot scope are documented and approved by the Groups/security owners.* ***Pass*** *→ Configure.*

Stage 2 — Configure the org-wide baseline

Make these **once**, tenant-wide, before touching individual groups. Each has a rollback (revert to the values recorded in Step 0.2).

Step 2.1 — Restrict who can create groups. In **admin.google.com** → **Directory** → **Groups** → **Settings** (org sharing settings), set group creation to **admins/allow-listed only** (or route all creation through the lifecycle intake). This stops uncontrolled sprawl at the source. *Rollback:* re-open creation to the previous value.

Step 2.2 — Set the restrictive default access baseline. In the same settings, set the **default access level** to the internal-only / restricted option and **disable "allow external members"** at the org default (external is enabled per-group only on `ext-*` groups in Stage 3). *Rollback:* restore the previous default access + external-members value.

Step 2.3 — Lock inbound external posting default. In **Apps** → **Google Workspace** → **Groups for Business** settings, ensure the default for **who can post from outside the organization** is **restricted** (owners approve; not "anyone on the internet"). *Rollback:* restore the recorded inbound-posting default.

Step 2.4 — Create the default owner group. Create `Security – Groups – Owners` (`sec-groups-owners@corp.example`) as a **Cloud Identity security group** and add the lifecycle owners as members. This group becomes the `OWNER` of governed groups.

> ***Gate 2 (baseline configured):*** *creation restricted, default access restrictive, external members off by default, inbound external posting restricted, owner group created. Verify with* `groups_settings_policy_helper.py check-baseline --domain corp.example`*.* ***Pass*** *→ Pilot.*

Stage 3 — Pilot (apply the model to the pilot scope)

Do everything below **only for groups in** `Pilot OU – Groups – Finance` first. Every helper runs **dry-run by default**; add `--apply` only inside the change window.

Step 3.1 — Re-type and re-name pilot groups. For each pilot group, decide its **type** and bring its **name/email prefix** to standard. Preview then apply:

```
python groups_naming_type_validator.py --domain corp.example --scope finance --json    # report drift
```

Rename to the convention in the console (or via the config helper) so every pilot group is, e.g., `Distribution – Finance – All Staff` / `dl-finance-all@corp.example`.

Step 3.2 — Apply per-type settings. For each pilot group, set its **Groups settings** to the type's row from Step 1.1. Dry-run first, then apply:

```
python groups_settings_policy_helper.py --group dl-finance-all@corp.example --type distribution --dry-run
python groups_settings_policy_helper.py --group dl-finance-all@corp.example --type distribution --apply
```

Repeat per pilot group and type. The helper only writes the fields defined for that type and prints a before/after diff.

Step 3.3 — Convert access-granting groups to security groups. Any pilot group used to grant access (e.g. `sec-finance-ap@corp.example`) must carry the **security label**. Create/label it as a **Cloud Identity security group** and re-point the access grants (Shared Drive membership, IAM) at it. *Rollback:* the prior group/label is retained until the re-point is validated (Stage 4), then removed.

Step 3.4 — Convert eligible groups to dynamic. For a pilot group whose membership maps to a directory attribute (e.g. all Finance users), create the **dynamic group** `Dynamic – Finance – All Finance` (`dyn-finance-all@corp.example`) with the CEL query (e.g. `user.organizations.exists(o, o.department == 'Finance')` — **verify the exact CEL syntax in your tenant**) and validate its computed membership **before** retiring the equivalent static group.

Step 3.5 — Fix ownership. Ensure every pilot group has the owner group `sec-groups-owners@corp.example` as an `OWNER` and ≥**2 owners total**; remove single-owner and orphaned states:

```
python groups_settings_policy_helper.py --group <group> --set-owner sec-groups-owners@corp.example --dry-r
```

Step 3.6 — Scope external membership. Remove external members from any non-`ext-` pilot group. For a legitimately external pilot group, rename it `External – ProjectX – Partner Collab` (`ext-projectx-partner@corp.example`), allow-list the partner domain, and confirm its settings match the External row. **This is high-impact** (removing external members can break a live collaboration) — do it in the change window with owners notified. *Rollback:* re-add the removed external member(s) from the assessment record if a break is reported.

> ***Gate 3 (pilot pass/fail):*** *for* `Pilot OU – Groups – Finance`, `groups_naming_type_validator.py` *reports* ***zero*** *naming/type drift,* `groups_membership_external_policy_checker.py` *reports* ***zero*** *out-of-policy external members and no over-permissioned security groups, every group has ≥2 owners, and dynamic membership matches expected.* ***Any FAIL → fix or rollback that change; do not expand.***

Stage 4 — Validate the pilot

Step 4.1 — Naming/type conformance. `python groups_naming_type_validator.py --domain corp.example --scope finance` → expect **PASS** (no drift).

Step 4.2 — Membership & external policy. `python groups_membership_external_policy_checker.py --domain corp.example --scope finance` → expect **zero** external members outside `ext-*`, no public settings, no over-permissioned `sec-*` groups.

Step 4.3 — Ownership & hygiene. Re-run `groups_inventory_hygiene_assessment.py --domain corp.example` filtered to Finance → expect **zero ORPHANED**, **zero STALE** in the pilot scope (or each stale group ticketed for retirement).

Step 4.4 — Access re-point verification. Confirm that access previously granted via a repurposed group now flows through the security group from Step 3.3 (a Shared Drive/IAM check) before removing the old grant.

Step 4.5 — Generate the pilot review report. `python groups_lifecycle_review_report.py --domain corp.example --scope finance --out pilot-review.html` → produces the owner-attestation review artifact `Review Report – Finance – Pilot`.

> ***Gate 4 (validation):*** *all Step 4 checks PASS and the review report is signed off by the pilot owners.* ***Pass*** *→ Expand.* ***Fail*** *→ remediate within the pilot; do not roll out.*

Stage 5 — Expand & operationalise

Step 5.1 — Roll out scope by scope. Repeat Stages 3–4 for each remaining scope (function/BU), one at a time, each with its own `Pilot OU – Groups – <Scope>`-style change window and gate. Never big-bang the whole estate.

Step 5.2 — Operationalise the lifecycle. Route all new-group requests through the intake; approvals enforce type/naming/owner/external at creation. Schedule the **hygiene sweeps** (`groups_inventory_hygiene_assessment.py`) and the **owner-attested reviews** (`groups_lifecycle_review_report.py`) on the cadence from Step 1.4.

Step 5.3 — Staged retirement (the documented rollback for a group's life). To retire a group:

1. **Suspend** — set posting to `NONE_CAN_POST` and remove it from any active access grant (keep the group + membership recorded).
1. **Hold** — leave suspended for the rollback window (e.g. 30 days); if mail/access breaks are reported, **re-enable** (rollback).
1. **Delete** — after the hold with no breaks, delete the group.

Never delete a group that still routes mail or grants access in a single step.

> ***Estate gate (steady state):*** *across all in-scope groups, naming/type validator and external/policy checker report PASS, the hygiene assessment shows no orphaned groups and all stale groups ticketed, and every group has an owner group and a review date. This is the ongoing definition of "governed."*

5. Testing & Validation

Each test states **what to run**, the **expected result**, and the **pass/fail** signal. Tests map to the gates in Section 3. Run the read-only tests any time; run write-verification tests after a change window. Reference the named artifacts (`Pilot OU – Groups – Finance`, `sec-groups-owners@corp.example`, etc.) from the Implementation Guide.

Test 1 — Baseline captured (Gate 0)

- **Run:** `python groups_inventory_hygiene_assessment.py --domain corp.example --json > baseline-groups.json`

and confirm the Admin-console settings from Step 0.2 are recorded.

- **Expect:** a JSON baseline with per-category counts (ORPHANED / STALE / EXTERNAL-EXPOSED / OVER-PERMISSIONED / NAMING-NONCONFORMANT) and the recorded rollback settings.

- **Pass:** baseline artifact + recorded settings both exist. **Fail:** either missing — do not proceed.

Test 2 — Org-wide baseline configured (Gate 2)

- **Run:** `python groups_settings_policy_helper.py check-baseline --domain corp.example`

- **Expect:** group creation restricted, default access restrictive, **allow external members = off** at org default, inbound external posting restricted, and `sec-groups-owners@corp.example` exists as a security group.

- **Pass:** all five report OK (exit 0). **Fail:** any item non-compliant (exit 1) — fix in Stage 2 before piloting.

Test 3 — Naming & type conformance (Gates 3/4)

- **Run:** `python groups_naming_type_validator.py --domain corp.example --scope finance`

- **Expect:** every pilot group's display name and email prefix match `<Type> – <Scope> – <Purpose>` / `dl-`/`sec-`/`collab-`/`dyn-`/`ext-`, and its settings match the declared type's policy row.

- **Pass: zero** drift findings (exit 0). **Fail:** any nonconformant name/prefix/type (exit 1) — rename/re-type in Step 3.1/3.2.

Test 4 — Membership & external-exposure policy (Gates 3/4)

- **Run:** `python groups_membership_external_policy_checker.py --domain corp.example --scope finance`

- **Expect:** no external members on any non-`ext-` group; no group with public join/post/view settings outside its type policy; no `sec-*` access group with over-open settings (e.g. `ALL_IN_DOMAIN_CAN_POST`).

- **Pass: zero** out-of-policy findings (exit 1 is used as a gate signal when findings exist). **Fail:** any finding — remediate in Step 3.6 (external) / 3.2 (settings).

Test 5 — Ownership & no-orphan (Gates 3/4)

- **Run:** re-run `groups_inventory_hygiene_assessment.py --domain corp.example` filtered to the pilot

scope; inspect ORPHANED and owner-count.

• **Expect:** every pilot group has ≥**2 owners** including `sec-groups-owners@corp.example`; **zero ORPHANED**.

• **Pass:** no orphaned/single-owner groups in scope. **Fail:** any — fix in Step 3.5.

Test 6 — Stale-group hygiene

• **Run:** the same assessment; inspect the STALE list for the pilot scope.

• **Expect:** no group flagged STALE (empty / no recent activity beyond the threshold) that is **not** already ticketed for staged retirement.

• **Pass:** stale list empty **or** every stale group has a retirement ticket. **Fail:** an untracked stale group — start Stage 5 retirement.

Test 7 — Dynamic membership correctness

• **Run:** compare the computed membership of `dyn-finance-all@corp.example` against the expected set (e.g. the directory query for `department=Finance`).

• **Expect:** computed membership equals the expected attribute-derived set (allowing for sync latency).

• **Pass:** membership matches. **Fail:** mismatch — check the CEL query and the source attributes (Requirement 2.4) before retiring the static equivalent.

Test 8 — Security-group access re-point

• **Run:** confirm access previously granted via a repurposed group now flows through the Cloud Identity **security group** from Step 3.3 (check a Shared Drive membership / IAM binding).

• **Expect:** access is granted **only** via the security group; the old grant is removable without loss.

• **Pass:** access verified via the security group. **Fail:** access still depends on the old group — do not remove it yet.

Test 9 — Lifecycle review report generation

• **Run:** `python groups_lifecycle_review_report.py --domain corp.example --scope finance --out pilot-review.html`

• **Expect:** `Review Report – Finance – Pilot` listing each group with type, owner group, member count, external count, last-activity, and next-review date for owner attestation.

• **Pass:** report generated and owners can attest. **Fail:** report empty/errors — check API scopes (Requirement 2.6).

Test 10 — Rollback rehearsal (high-impact changes)

- **Run:** in the change window, revert one representative change (re-open a setting to the Step 0.2 value, or re-add a removed external member from the baseline record), confirm the group behaves as before, then re-apply the governed setting.
- **Expect:** rollback restores prior behaviour cleanly; re-apply restores the governed state.
- **Pass:** both directions work. **Fail:** rollback doesn't restore mail flow/access — investigate before expanding (Section 6).

6. Troubleshooting

Common failure modes when standing up and running Google Groups governance, with concrete resolutions. Symptoms reference the named artifacts and tools from the Implementation and Testing sections.

Auth, scopes & API access

Symptom — a tool exits with `403 insufficient permissions` / `insufficientPermissions` / `Not Authorized to access this resource`.

- The ADC/service-account identity lacks the scope for that call. Assessment/validation need the **read-only** scopes (Requirement 2.6); `groups_settings_policy_helper.py --apply` needs the **write** scopes.
- For a **delegated service account**, the scopes must be authorized in **Admin console → Security → API controls → Domain-wide delegation** against the SA's client ID, *and* the SA must impersonate an admin who holds the Groups/Cloud Identity privilege. A missing DWD scope is the most common cause.
- Confirm the **Admin SDK API** and **Cloud Identity API** are enabled on the project.

Symptom — `groupssettings` calls fail but Directory calls succeed.

- The **Groups Settings API** is a separate service (`groupssettings/v1`) with its own scope (`apps.groups.settings`). Enable it and add the scope; a read of settings needs the scope even though Directory reads worked.

Symptom — `ImportError` / `pip install google-api-python-client google-auth`.

- The Google client libraries aren't installed in the active interpreter. Install them; the tools import lazily so `--help` still works without them.

Naming/type validator failures (Gate 3/4)

Symptom — a group is flagged `email prefix off-standard`.

- Its local-part doesn't start with `dl-`/`sec-`/`collab-`/`dyn-`/`ext-`. Decide its type and rename the

address to the convention (Step 3.1). Renaming an email address changes where mail is delivered — do it in the change window and add an alias for the old address if it was in active use.

Symptom — a group is flagged `display name not '<Type> – <Scope> – <Purpose>'`**.**

- The validator expects a three-part, en-dash-separated name. Rename in **Directory → Groups →** (group)

→ group name**. If your standard legitimately uses a hyphen instead of an en dash, adjust the name to the exact separator your policy defines (the tool accepts either dash, but the string must have three parts).

Symptom — `whoCanPostMessage=… not in policy for type …`**.**

- The group's settings don't match its type row (Step 1.1). Apply the type policy:

`groups_settings_policy_helper.py apply-settings --group <g> --type <type> --dry-run`, inspect the diff, then `--apply`.

Settings that "won't stick" / apply has no effect

Symptom — after `apply-settings --apply`**, a re-check shows the old value.**

- Some Groups settings are **overridden by an Admin-console org-wide default or an OU policy** (e.g.

external members disallowed org-wide will keep `allowExternalMembers` effectively off regardless of the per-group value). Reconcile the org baseline (Stage 2) with the per-group intent, or set the per-group override where the console allows it — **verify in your tenant** which setting wins.

- A **deprecated/renamed enum**: if the API accepts a value but it doesn't take effect, the field or

value may have changed for your tenant version. Check the current Groups Settings API reference and adjust the value in the type policy.

External-member removal breaks mail flow or collaboration

Symptom — after scoping external membership (Step 3.6), an external partner stops receiving a distribution, or a shared inbox thread breaks.

- Removing an external member from a group that was silently acting as an external distribution point

cuts that delivery. **Rollback:** re-add the removed member from the `Baseline – Groups Estate – Assess` record. Then decide the correct home: if the external relationship is legitimate, move it to a properly typed `ext-` group with an allow-listed partner domain rather than leaving external members on an internal group.

Symptom — a legitimate external group can't accept partner mail after tightening.

- Confirm the group is typed `ext-`, `allowExternalMembers=true`, the partner domain is allow-listed,

and inbound external posting is permitted for that group. The org-wide inbound-posting restriction (Step 2.3) can block external senders even when membership is correct — grant the exception at the group level.

Dynamic-group membership is wrong or empty

Symptom — a `dyn-` group has fewer/more members than expected, or none.

- The **CEL query** doesn't match, or the **source attribute is empty/stale** for the users concerned

(Requirement 2.4). Verify the attribute (`Directory → Users`) is populated and current for a sample, and that the CEL expression references the right attribute path — **verify the exact CEL syntax in your tenant**, which is version-dependent.

- **Sync latency:** dynamic membership recomputes after directory changes; allow for propagation before

concluding the query is wrong. **Never retire the equivalent static group until Test 7 confirms the computed membership matches.**

Ownership / orphaned groups

Symptom — the hygiene assessment keeps reporting ORPHANED groups.

- The `OWNER` is a **suspended/deleted user**, or the group has no owner at all. Add the owner group

`sec-groups-owners@corp.example` as `OWNER` (`groups_settings_policy_helper.py set-owner …`) and ensure ≥2 owners. Prefer the owner *group* so a future departure doesn't re-orphan it.

Symptom — `set-owner --apply` fails with `memberKey not found` or a role error.

- The owner group must **exist** first (Step 2.4) and be resolvable. Create

`sec-groups-owners@corp.example` as a Cloud Identity security group before assigning it as OWNER.

Security-group / Cloud Identity issues

Symptom — a group can't be used in an IAM binding or a context where a security group is required.

- Plain Groups-for-Business groups aren't security groups. The group must carry the **security label**,

which requires **Cloud Identity Premium** (Requirement 2.2) and creation/labelling as a **security group**. Confirm the subscription, then create/label it — **verify in your tenant** whether an existing group can be converted or must be recreated.

Retirement pitfalls

Symptom — deleting a group broke mail delivery or an access grant.

- A group was deleted before it was fully de-referenced. Follow **staged retirement** (Step 5.3):

suspend (stop posting + remove from access grants) → **hold** for the rollback window → **delete** only after no breaks are reported. If a break occurs during the hold, **re-enable** (rollback). If the group is already deleted, recreate it with the same address and re-point the affected grants from the baseline record.

Rate limits / large estates

Symptom — sweeps slow down or return `rate limit exceeded` / `quotaExceeded` on a large tenant.

• The Admin SDK enforces per-project quotas. Scope runs with `--scope` (per function/BU) and `--max` to

stay under quota, run sweeps off-peak, and add backoff/retry if you extend the tools. The assessment is safe to run incrementally — it's read-only and always exits 0.

7. Best Practices

Durable practices for keeping Google Groups governed as the estate grows. These outlast the initial rollout — treat them as the steady-state operating standard.

Model & typing

• **Type every group; type drives everything.** Never create a group without deciding its type

(Distribution / Security / Collaboration / Dynamic / External). The type sets the default settings, the review cadence, and whether external members are even allowed. An untyped group is an ungoverned group.

• **The name is the contract.** Enforce `<Type> – <Scope> – <Purpose>` and the email prefix at

creation (via the intake), not after the fact. A machine-readable name lets the validator gate the whole estate cheaply — the naming standard is what makes governance auditable rather than aspirational.

• **Security label anything that grants access.** If a group is ever added to a Shared Drive, calendar,

IAM binding, target audience, or SSO app, make it a **Cloud Identity security group**. The immutable label stops it being silently repurposed and is a prerequisite for IAM use.

Ownership & membership

• **Never a single owner; prefer an owner *group*.** Require ≥**2 owners**, and make the `OWNER` a group

(`sec-groups-owners@…`) rather than named individuals so ownership survives departures. Orphaned groups are the most common and most dangerous defect — the hygiene sweep must catch them continuously.

• **Membership by group, not by person.** Nest groups instead of maintaining long individual-member

lists. Access then lives in one auditable place and follows the directory source of truth.

• **Prefer dynamic membership where an attribute fits.** For department/location/cost-centre-shaped

groups, a dynamic group eliminates the "forgot to remove them" error class — but only after confirming the source attribute is trustworthy (Requirement 2.4). A dynamic group over bad attributes is worse than a static one.

Exposure & trust boundary

• **Restrictive by default, opened deliberately.** Keep the org-wide baseline closed (creation

restricted, external members off, inbound external posting restricted) and open exposure only per-group on explicitly `ext-` groups with allow-listed partner domains.

- **Treat "external members" and "public settings" as findings, not features.** Any external member on a

non-`ext-` group, any world-readable archive, or any open posting outside policy is a defect the checker flags and an owner must justify or fix.

- **Access groups don't accept mail.** Security/access groups should post `NONE_CAN_POST` — if an access

principal is also an open mailing list, that is over-permissioning waiting to be abused.

Lifecycle & hygiene

- **Route all creation through the intake.** Uncontrolled self-service group creation is the source of

sprawl; restrict creation and make the request → approve path the only way in, enforcing type/naming/owner/external at the door.

- **Review on a cadence proportional to risk.** Security and external groups reviewed quarterly;

distribution and dynamic groups annually. Owner attestation on the generated review report is the evidence.

- **Retire in stages, never in one destructive step.** Suspend (stop posting / remove from access) →

hold for a rollback window → delete. Deleting a group that still routes mail or grants access breaks both silently.

- **Sweep continuously, not just at rollout.** Schedule the hygiene assessment so orphaned/stale/

external-exposed/over-permissioned groups surface between reviews, not only during them.

Automation & change safety

- **Read-only first, dry-run always.** Run assessments and validators freely; run any write helper in

dry-run and inspect the diff before `--apply`, and only `--apply` inside a change window.

- **No secrets in code, least-privilege scopes.** Auth via ADC or a delegated service account; grant the

read-only scopes broadly and the write scopes only to the identity that runs `--apply`. Never commit a key file.

- **Gate expansion on the pilot.** Do not roll a change past the pilot scope until the naming/type

validator and external/policy checker both PASS and the review report is attested — one scope at a time, never a big-bang across the estate.

8. Deliverables Summary

This solution delivers a complete, self-guided package to bring Google Groups under a real management and governance model — the documentation to design and run it, and five Python tools to assess, configure, and continuously validate the estate.

Documentation

• **Solution Overview** — the Groups governance model: the five/six design decisions (types, naming, ownership, dynamic vs. security groups, external/posting policy, lifecycle & hygiene), the reference target state, benefits, honest limitations, and the deployment model.

• **Requirements Guide** — the solution-specific prerequisites (Groups/Cloud Identity admin rights, Cloud Identity Premium for dynamic/security groups, the type catalogue & naming standard, the directory attribute source of truth, lifecycle owners & intake path, and least-privilege API scopes), building on the shared Google global requirements.

• **Implementation Guide** — ordered, named steps following Assess → Design → Configure → Pilot → Validate → Expand, with per-type settings, pass/fail gates at every stage, and a staged-retirement rollback.

• **Testing & Validation** — ten tests mapped to the gates, each with an expected pass/fail signal.

• **Troubleshooting** — the common failure modes (API scope/permission errors, settings that won't stick, dynamic-membership mismatches, external-member removal breaking mail flow, security-label and retirement pitfalls) and their resolutions.

• **Best Practices** — the durable steady-state operating standard for keeping Groups governed as they grow.

The 5 Python tools

All tools use the Google **Admin SDK Directory (Groups / Members / Users)**, **Groups Settings**, and **Cloud Identity Groups** APIs. All are **safe-by-default** (read-only, or dry-run unless `--apply`), carry **no credentials or keys in code** (auth via ADC / a delegated service account), and expose `--help`.

1. `groups_inventory_hygiene_assessment.py` *(read-only)* — the Stage-0 baseline. Enumerates every

group and classifies each into **ORPHANED / STALE / EXTERNAL-EXPOSED / OVER-PERMISSIONED / NAMING-NONCONFORMANT**, with counts and samples. Produces `Baseline – Groups Estate – Assess`.

1. `groups_naming_type_validator.py` *(read-only gate)* — asserts every group matches the `<Type> – <Scope> – <Purpose>` / prefix naming standard **and** the declared type's settings policy. Exits non-zero on any drift so change control can gate on it (Gates 3/4).

1. `groups_membership_external_policy_checker.py` *(read-only gate)* — enforces the trust boundary:

no external members on non-`ext-` groups, no public join/post/view outside type policy, no over-permissioned security groups. Exits non-zero on any violation (Gates 3/4).

1. `groups_lifecycle_review_report.py` *(read-only, writes a local report)* — generates the

owner-attested `Review Report – <Scope> – <Stage>` (type, owners, member/external counts, last-activity, next-review by cadence) and flags groups DUE/OVERDUE for review.

1. `groups_settings_policy_helper.py` *(dry-run default; the only write tool)* — the config helper:

`check-baseline` (read-only Gate-2 org baseline check), `apply-settings` (write a type's Groups settings to a group), and `set-owner` (ensure the owner group holds `OWNER`). Every write is dry-run with a before/after diff unless `--apply`.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

groups_inventory_hygiene_assessment.py

groups_lifecycle_review_report.py

groups_membership_external_policy_checker.py

groups_naming_type_validator.py

groups_settings_policy_helper.py